5. Implementation

5.0 System Architecture and Frame-Rate Independence

5.0.1 Objectives and Motivations

Before any of the gameplay-specific systems described in the sections below could be built, the game needed a predictable execution loop. Two concerns drove the top-level architecture: first, that the same source code must serve radically different screens—a main menu, three distinct levels, a shop, a pause overlay, a victory screen, and a game-over screen—without those subsystems interfering with each other; and second, that the game must behave consistently across hardware, so that a player on a 144 Hz gaming laptop and a player on a 30 Hz budget browser see the same character speed and the same enemy behaviour. Both concerns are invisible when they work and catastrophic when they don't, so getting them right early was a prerequisite for every mechanic built later.

5.0.2 State-Flag Dispatch and Delta-Time Scaling

Scene control is implemented through a small set of global boolean flags—start, pause, gameover, end, together with per-level overlay flags such as showInstructions and showMiniMap—that gate which subsystems run each frame. The draw() function itself is deliberately minimal:

function draw() { let dt = deltaTime / 1000; if (dt > 1) dt = 0; act(dt); drawGame(); }

All game logic lives in act(dt) and short-circuits immediately when pause is true, while drawGame() dispatches to the appropriate UI routine based on the flag combination (e.g., if (pause) { if (start) drawStart(); else if (end) drawEnd(); ... }). This separation prevents enemy updates from bleeding into pause screens—a class of bug that dogged our earliest prototypes—and makes adding new scenes a matter of extending the flag vocabulary rather than rewriting the loop. Because drawFog(), drawMiniMap(), drawHud(), and so on are simple functions guarded by their own flags, every level file reuses the same skeleton while defining its own subsystems. Delta-time decoupling sits on top of this skeleton. p5.js exposes deltaTime in milliseconds; we convert to seconds (dt = deltaTime / 1000) so that every downstream calculation is expressed in intuitive units—player speed is 120 px/s, boss speed is 36 px/s, shoot cooldown is 1.6 s—rather than frame-count magic numbers. All movement then multiplies by dt (player.top -= 120 * deltaTime), as do all countdown timers (shootTimer -= dt, playerInvulnTimer -= dt, etc.), guaranteeing frame-rate independence. A one-line guard, if (dt > 1) dt = 0, handles the edge case of the browser tab losing focus: on return the accumulated delta can exceed one second, which without the clamp would teleport the player clear across the maze in a single "catch-up" frame. Collision response rides on the same pattern. Movement is applied per axis, and after each axis move the player's hitbox is tested against every wall; on intersection, the relevant edge is snapped to the wall (player.right = w.left, player.top = w.bottom, etc.) using Rect setters that propagate to the opposite edge automatically. Because the axes are handled independently, the sliding behaviour along wall corners emerges for free rather than requiring special-case diagonal logic.

5.0.3 Role as a Foundation for the Rest of This Chapter

Every system described in the following sections assumes this foundation. The fog buffer in §5.1 is redrawn fresh every frame inside drawGame() because drawFog() is a flag-gated call like any other UI routine. The spawn loops in §5.2 can afford their 300–400 retry iterations because they run once during setup(), not per-frame. The boss AI in §5.3 scales its chasing velocity by dt and its shoot timer by dt using exactly the same pattern as the player—meaning the boss is guaranteed to behave the same way on every machine, and its tuning constants are real-world seconds rather than opaque magic numbers.

5.1 Atmospheric Vision Masking and Combat Synchronisation

5.1.1 Objectives and Motivations

A defining feature of our Dark Maps mode is fog-of-war: the player sees only a small radius around the character and must locate a torch to extend visibility. This mechanic had to feel atmospheric and performant, but it also had to remain fair—the combat system must not allow the player to interact with entities hidden inside the fog, and enemies must not aggro through walls of darkness they appear to lie beyond. Synchronising what is rendered with what is mechanically valid was therefore a correctness requirement, not merely a visual concern.

5.1.2 Off-Screen Buffer Rendering and Distance-Capped Targeting

Fog-of-war is rendered using a secondary off-screen p5.Graphics buffer, initialised once at setup as fogLayer = createGraphics(width, height) and reused every frame. The drawFog() routine first fills the buffer with a semi-opaque navy (fill(10, 12, 18, 185)) to establish the dark layer, then—if hasLight is true—punches a transparent "hole" around the player by calling fogLayer.erase() before drawing a circle of diameter fogRadiusWithLight * 2 (with fogRadiusWithLight = 165 px) centred on the player's screen coordinates. The buffer is finally composited over the main canvas in a single image(fogLayer, 0, 0) call. Because the buffer is cleared and fully redrawn each frame rather than mutated in place, performance remains stable even when additional light sources are introduced, and we avoid per-pixel operations on the primary rendering surface entirely. The more subtle challenge was not drawing the fog but honouring it mechanically. The omnidirectional shooting system includes auto-lock targeting via findAutoTarget(px, py, range), which iterates over enemies[] and returns the nearest live enemy within range. A naïve implementation would use an unlimited range, allowing the crossbow to lock onto enemies the player could not possibly see—trivialising hidden encounters and breaking the atmosphere we had just spent effort rendering. To close this exploit, we cap the targeting range to closely match the fog's visibility radius of 165 px: 210 px in Level 2 (crossbow) and 220 px in Level 3 (gun, whose lamp grants marginally larger situational awareness). In both cases the small surplus over 165 accounts for the feathered edge of the halo and feels intuitive to players. Enemies outside this radius are excluded from the candidate list even though they exist in the world state. Crucially, a Euclidean distance test (d = sqrt(dx*dx + dy*dy)) is used rather than a

bounding-box check, so the "lock zone" is genuinely circular and cannot be gamed by off-axis corner exploits.

5.1.3 Reflections and Extensibility

Aligning the rendering layer and the game-logic layer through a shared notion of visibility proved the cleanest way to keep both systems honest. The single createGraphics buffer pattern is also cheap enough that it could be extended with additional transparent holes per active torch or lamp without architectural change—each extra light source is a one-line fogLayer.circle() call inside the erase block. Future work could generalise this into directional cones (flashlights), fog-piercing consumables sold in the shop, or stealth mechanics that only register the player once the player enters an enemy's visibility window. Exposing the visibility radius as a named constant rather than a magic number has already made it trivial to tune playtest balance without grep-hunting through subsystems.

5.2 Procedural Constraint-Based Spawning

5.2.1 Objectives and Motivations

To keep each playthrough feeling fresh and to prevent memorisation from trivialising later attempts, all pickups (chest, lamp, ring, crossbow) enemies and portal endpoints are spawned procedurally within each level. Randomness alone, however, produces unacceptable outcomes: an item placed inside a wall is uncollectable, an enemy overlapping the player spawn is an instant death and a pickup placed behind an already-placed item creates a visually ambiguous pile. Balancing procedural variety with absolute solvability was the central challenge of this subsystem.

5.2.2 Grid Snapping and Bounded Resampling with Exclusion Lists

In Level 2 and Level 3 all candidate positions are first passed through snapToGrid(value) (defined as floor(value / blockSize) * blockSize) which aligns them to the 32-pixel grid and eliminates a whole class of near-miss sub-pixel overlaps that plagued our earliest prototypes. Level 1 achieves the same alignment through an explicit modulo check inside its spawn loop (while (boxX % blockSize !== 0 || ...)) producing the same guarantee without a dedicated helper. On top of this, a generic sampling routine—createItemInStartView(avoidList) for opening-area spawns and createItemInMidArea(avoidList) for mid-map spawns—iterates up to 300–400 times, each iteration drawing fresh random coordinates within a region-appropriate bounding box and rejecting any candidate that (a) overlaps a wall tile (intersectsWall(item)), (b) intersects the player's hitbox or (c) intersects any object in the supplied avoidList. Placement order threads the exclusion list forward: in Level 2, createBox([]), then createLight([box]), then createRing([box, lightItem]) so each new item automatically avoids every prior one without the caller having to manage state. If the loop exhausts its retry budget—rare but possible in dense mazes—the routine returns a hard-coded safe fallback (new Rect(128, 128, blockSize, blockSize, true)) so the game never fails to start. Enemy placement uses a similar but stricter loop: each candidate must avoid walls, the player, and every previously-placed

pickup and the exit door. Because enemies are placed only after all pickups are finalised, a simple while loop that re-samples on any collision is sufficient, and the exclusion list is effectively the entire set of critical game objects. Portal placement in Level 3 uses a two-stage strategy: first 100 attempts jittered around four hand-picked target coordinates (so the portal network roughly preserves its intended topology), then—if any of those four fail—a fallback 200-attempt pass across the full map, constrained to lie outside the player's starting view (isInStartView(px, py)) to avoid a portal appearing inches from the spawn.

5.2.3 Reflections and Extensibility

Decoupling level content from level layout via this constraint-based spawner means that new item types or enemy archetypes can be added by plugging into the existing createItemIn* helpers and extending the avoidList threading—no hand-placement required. The bounded-retry pattern is also self-documenting about its own failure modes: the fallback position and the iteration cap make the worst-case behaviour explicit, which made debugging "impossible seed" edge cases straightforward during playtest. Future work could extend this into semantic constraints (e.g., guaranteeing at least one torch lies within a Manhattan-distance of 8 tiles from the start in Dark Maps, or weighting enemy density by district theme) without touching the core sampling loop, since those would simply be additional predicates evaluated inside the candidate-rejection step.

5.3 Boss AI: State-Aware Pursuit and Fan-Shot Attack

5.3.1 Objectives and Motivations

The final boss encounter in Level 3 needed to feel distinctly different from the wandering enemies the player had faced up to that point. Random movement and single-shot projectiles—fine for background threats, would have made for a pretty underwhelming finale. Instead, the goal was to design behavior that actively hunted the player, forced them to keep moving and punished anyone who tried to stand still, all while fitting smoothly into the existing projectile system used by regular enemies.

5.3.2 Distance-Gated Chasing and Three-Shot Fan Spread

The Boss class maintains two pieces of state: health (bossMaxHp = 180) and a countdown timer (shootTimer, reset to bossShootCooldown = 1.6 seconds after each volley). Every update(dt) tick computes the vector from the boss to the player and its Euclidean distance, then branches on two distance thresholds that together define the boss's behavioural envelope: Chase phase (dist > 130): the boss normalises the player-vector and moves along it at bossSpeed = 36 px/s scaled by dt. A simple write-and-revert collision check attempts the move, then reverts left/top to their pre-move values if any wall intersection is detected. This keeps the boss physically bounded by the same tile grid the player uses, without any separate pathfinding subsystem. Attack phase (dist < 340): the shoot timer ticks down; when it expires, the boss fires fireFanShot(px, py) and resets the timer. The fan shot computes baseAngle = atan2(dy, dx) toward the player and emits three HostileProjectile instances at baseAngle - 18°, baseAngle, and baseAngle + 18°—a total spread of 36 degrees. Each projectile uses precomputed velocity (vx = cos(a) * 105, vy = sin(a) * 105) rather than axis-aligned movement, enabling diagonal trajectories that the existing HostileProjectile

update step handles uniformly with all other enemy shots. The two phases overlap: within 130–340 px, the boss both chases and fires, which is where the fight is most pressured. Beyond 340 px the boss chases in silence; inside 130 px it stops chasing but keeps firing, meaning a player who tries to juke-and-melee encounters a wall of projectiles at point-blank range. Contact damage (bossContactDamage = 16) on intersection with the player's hitbox closes the final loophole and rewards distance management.

5.3.3 Reflections and Extensibility

What made this implementation tractable was that no new subsystems were needed: movement reused the existing axis-based collision pattern; projectiles reused HostileProjectile, already written for mob enemies; targeting reused the atan2-based angle computation already used by the player's omnidirectional crossbow. The boss is therefore roughly 90 lines of new code rather than a parallel engine, and its distance thresholds are tuned as named constants (bossSpeed, bossShootCooldown, etc.) at the top of the file, making balance passes quick. The same skeleton would naturally support phase-two behaviours (swap fan-shot for a circular burst below 50% HP), teleport abilities (lean on the portal-search helper from Section 5.2), or multi-target spread angles (change the [−spread, 0, +spread] list to five or seven entries)—all as incremental extensions rather than rewrites.